



DFX4ML-ARCH (Progress Update 8-6-26)

Omitted for blind review

CERN, CH-1211 Geneva, Switzerland

Keywords: FPGA, Partial Reconfiguration, Dynamic Function Exchange, PYNQ, Xilinx

Summary

Disclaimer: Claude (Anthropic) was used as an AI assistant in the preparation of this report, including drafting and editing text, generating figures, and writing supporting code. All results, figures, and conclusions were reviewed and verified by the author.

Contents

1	Architecture Upgrade	3
1.1	Dedicated Memory Transfer Ports	3
1.2	DFX streamer reusable in multiple region	3
2	Experiment Setup	4
2.1	Model Architecture	4
2.2	Model Partitioning	6
2.2.1	Four-Part Partitioning	6
2.2.2	Two-Part Partitioning	8
2.3	DFX Streamer Management for current experiments	9
2.3.1	DFX Streamer allocation algorithm	9
2.3.2	4-sub model DFX streamer allocation	12
2.3.3	2-sub model DFX streamer allocation	13
3	Experiment Result	14
3.1	Resource Usage	14
3.1.1	Four-Part Scheme	14
3.1.2	Two-Part Scheme	16
3.2	performance	19
3.2.1	Four-Part Scheme	19
3.2.2	Two-Part Scheme	19

3.2.3	Combined Comparison	21
4	Why DFX4ML? and Its Position	23
5	Resources	24
6	Next Step	24

1 Architecture Upgrade

1.1 Dedicated Memory Transfer Ports

Since we expect that the region may reconfiguring while inference data may also transferring, we then upgrade the system's memory transfer port to have its own dedicated port connected to the processor(PS), as shown in Figure 1.

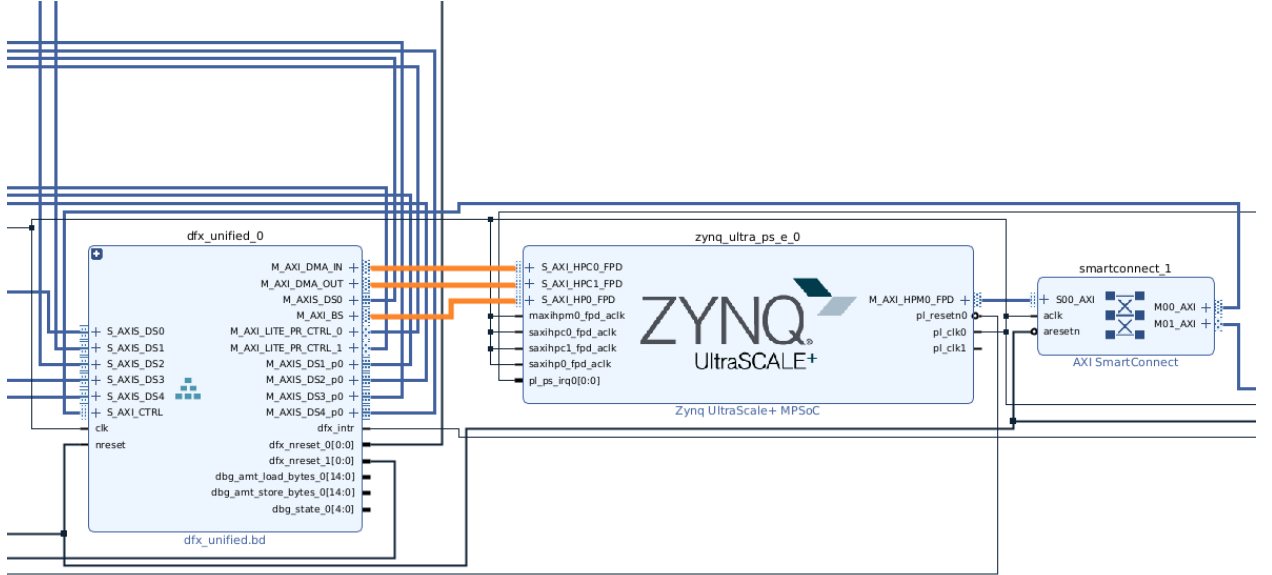


Figure 1: Memory access upgrade: a dedicated PS port for inference data transfer.

For data transferring, there are 3 dedicated port as followed:

1. M_AXI_DMA_IN : for DFX streamer 0 to load data to the partial region for ML inferencing.
2. M_AXI_DMA_OUT : for DFX streamer 0 to store data to the partial region for ML inferencing.
3. M_AXI_BS : for reconfiguring the partial region.

1.2 DFX streamer reusable in multiple region

A single DFX streamer can be shared across multiple reconfigurable regions, loading data into each region as needed, as shown in Figure 2. However, the streamer can only transfer to single region at a time.

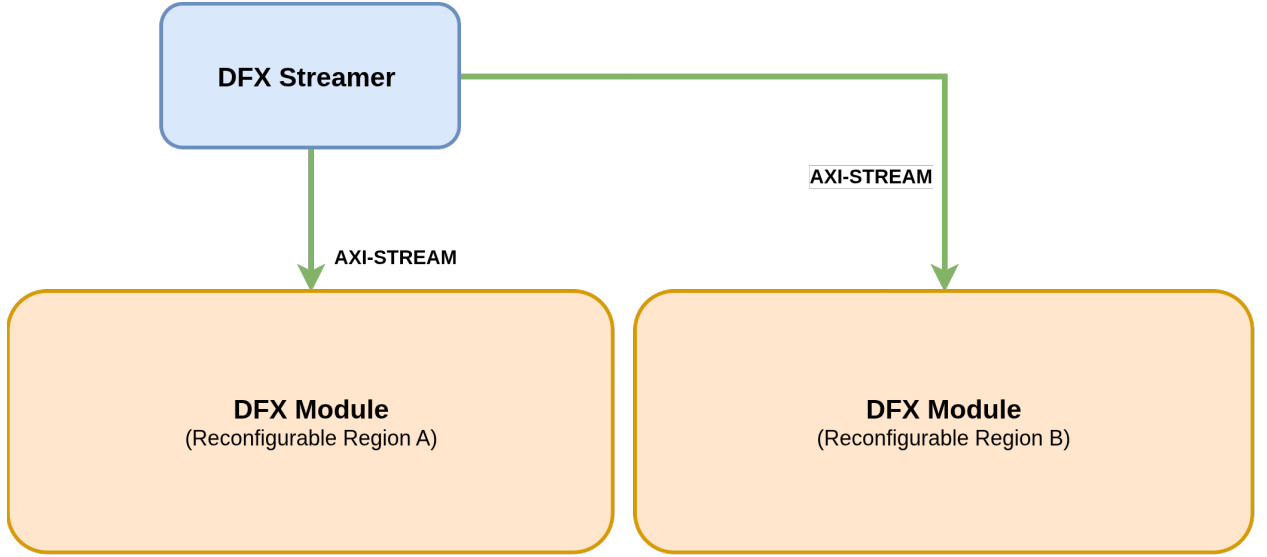


Figure 2: A single DFX streamer reused to load data across multiple reconfigurable regions.

2 Experiment Setup

2.1 Model Architecture

To test the performance in the practical use-case, we apply the U-Net architecture for this section, as shown in Figure 3.

Full Model — Skip-Connection Conv NN (U-Net 16 bits precision)

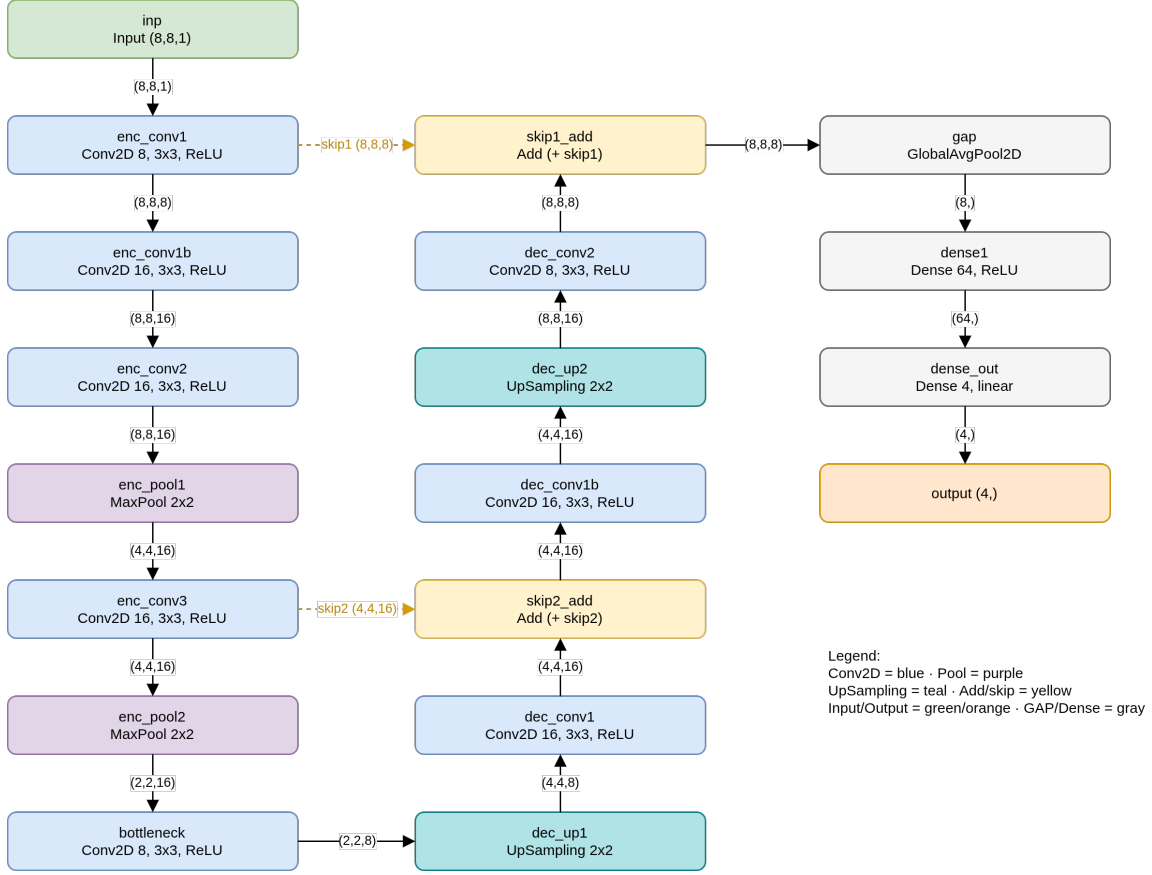


Figure 3: Skip-connection convolutional network (U-Net style) used for the experiment.

The model takes an $(8,8,1)$ input and produces a length-4 output vector. Every convolutional layer uses a 3×3 kernel with **same** padding and ReLU activation, so the spatial size changes only at the pooling and upsampling layers. The network is partitioned into four sequential parts, which map naturally onto the reconfigurable modules:

1. **Encoder-A:** `enc_conv1` (Conv2D 8) produces the first skip tensor `skip1` $(8,8,8)$; `enc_conv1b` and `enc_conv2` (Conv2D 16) refine the features, and `enc_pool1` (MaxPool 2×2) downsamples to $(4,4,16)$.
2. **Encoder-B + Bottleneck:** `enc_conv3` (Conv2D 16) produces the second skip tensor `skip2` $(4,4,16)$; `enc_pool2` downsamples to $(2,2,16)$, and the `bottleneck` (Conv2D 8) compresses to $(2,2,8)$.
3. **Decoder-A:** `dec_up1` (UpSampling 2×2) restores $(4,4,8)$; `dec_conv1` (Conv2D 16) is fused with `skip2` by element-wise addition at `skip2_add`, followed by `dec_conv1b` (Conv2D 16) $\rightarrow (4,4,16)$.
4. **Decoder-B + Head:** `dec_up2` restores $(8,8,16)$; `dec_conv2` (Conv2D 8) is fused with `skip1` by element-wise addition at `skip1_add` $(8,8,8)$. The classification head

then applies `gap` (GlobalAveragePooling2D), `dense1` (Dense 64, ReLU), and `dense_out` (Dense 4, linear) to yield the final (4,) output.

Note that the two skip connections fuse features by *addition* rather than concatenation, so the channel counts of the encoder and decoder tensors must match at each fusion point. The model is trained with the Adam optimizer and a categorical cross-entropy loss.

2.2 Model Partitioning

We set up two use-case scenarios: first, we divide the model into four parts, and second, we divide it into two parts.

2.2.1 Four-Part Partitioning

In the four-part scheme, the model is split at the boundaries of the four sequential parts and mapped onto two physical reconfigurable regions over four reconfiguration phases, as shown in Figure 4. The part 1 and 3 are loaded into *region 0*, and the part 2 and 4 into *region 1*; the two regions are reconfigured alternately, so that one region can execute its current part while the other is being reconfigured for the next.

4 Sub Models — Skip-Connection Conv NN (U-Net 16 bits precision)

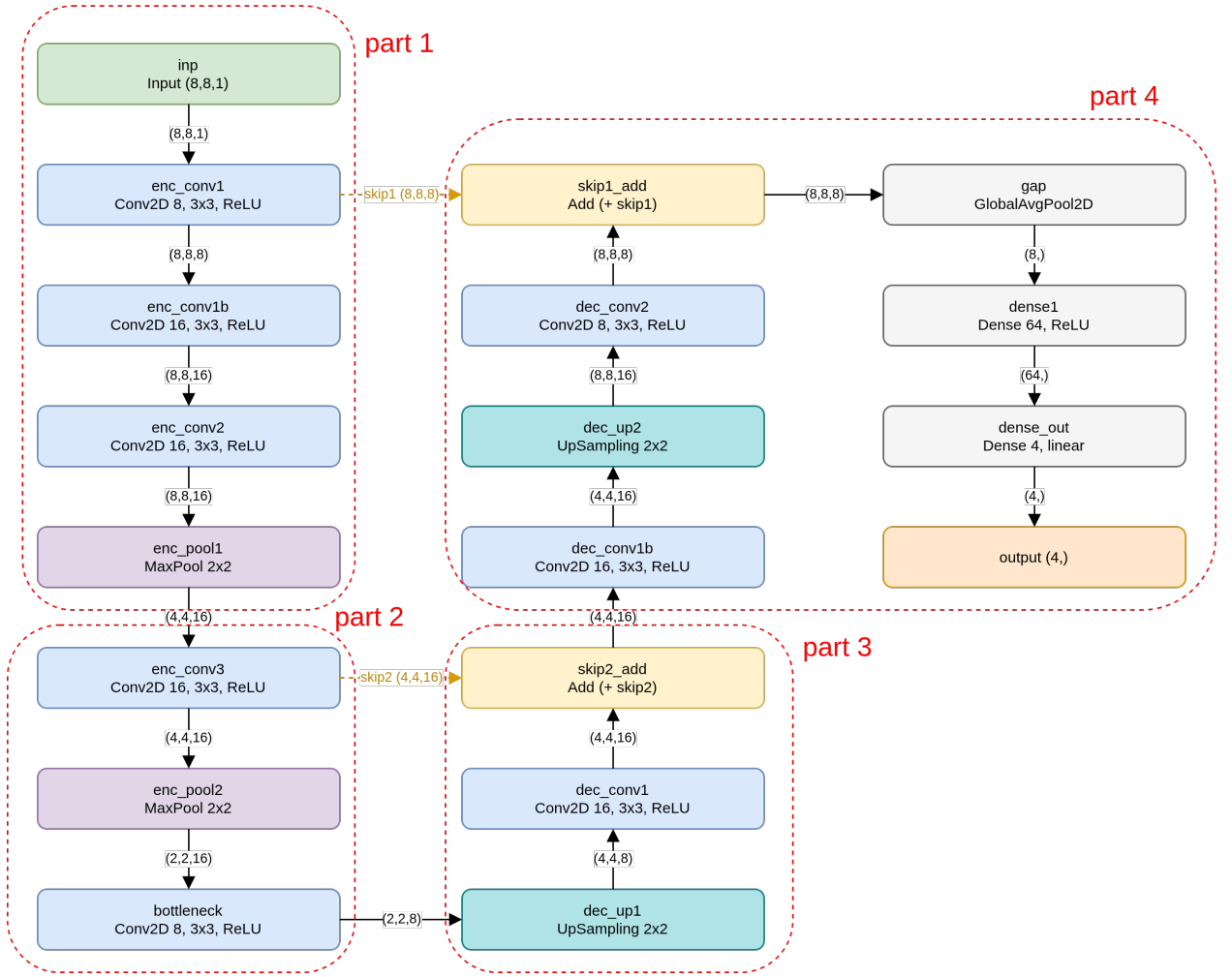


Figure 4: Four-part partitioning scheme: colored boxes are the DFX regions/phases, solid purple arrows are the main inter-partition streams, and dashed gold arrows are the skip streams.

The four parts map to the physical regions and reconfiguration phases as shown in Table 1.

Table 1: Mapping of the four model parts to physical regions and reconfiguration phases.

Part	Stage	Region	Phase
1	Encoder-A	0	0
2	Encoder-B + Bottleneck	1	1
3	Decoder-A	0	2
4	Decoder-B + Head	1	3

Region 0 hosts Parts 1 and 3 while region 1 hosts Parts 2 and 4, so each region is reused twice across the pipeline.

2.2.2 Two-Part Partitioning

In the two-part scheme, the model is split at the bottleneck into two halves, as shown in Figure 5. Half A holds the entire encoder and bottleneck (Parts 1–2), while Half B holds the decoder and classification head (Parts 3–4). Since the split is a single boundary, the main bottleneck stream and both skip streams all cross from Half A to Half B.

2 Sub Models — Skip-Connection Conv NN (U-Net 16 bits precision)

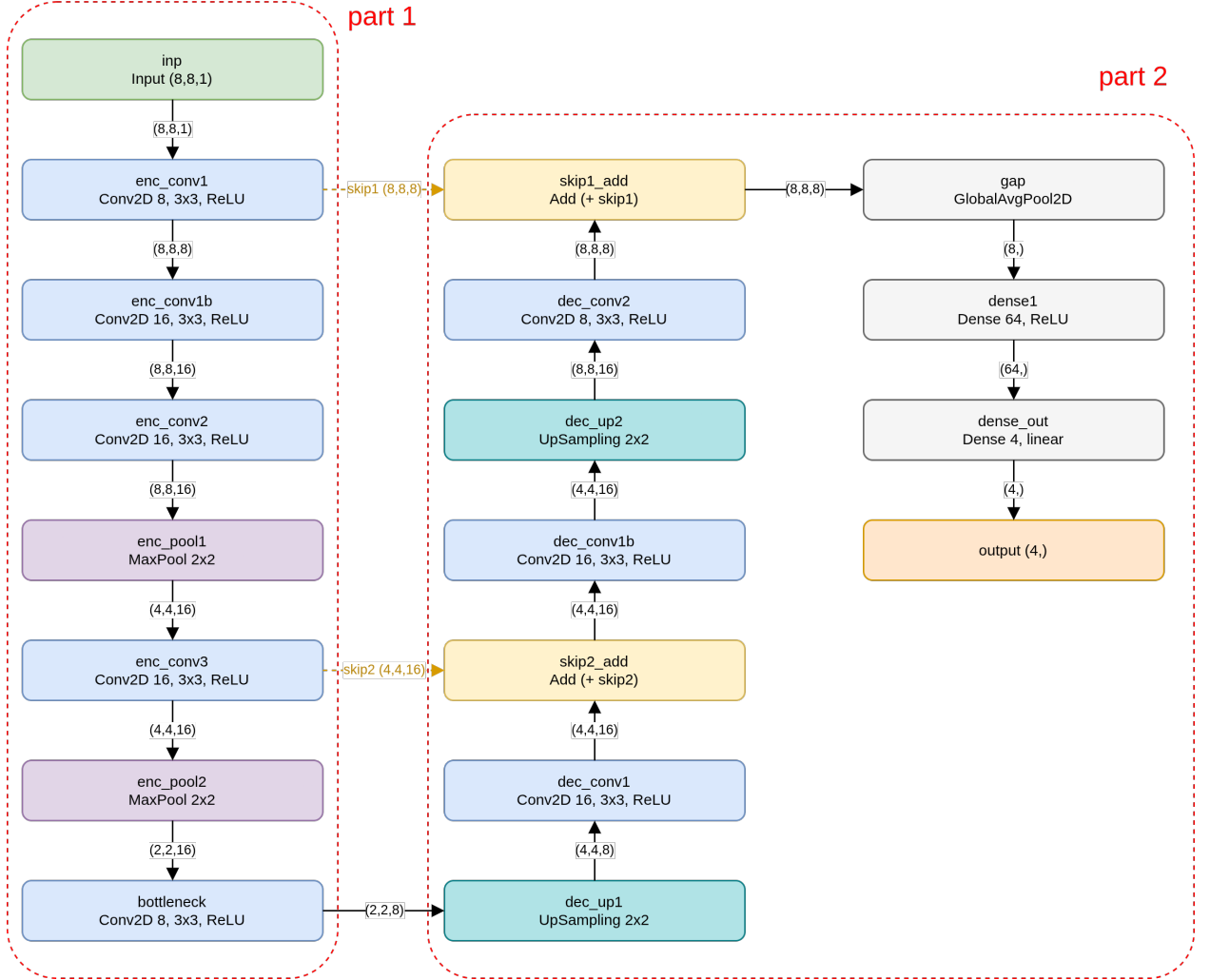


Figure 5: Two-part partitioning scheme: the model is split at the bottleneck into Half A (encoder + bottleneck) and Half B (decoder + head).

The two halves map to the physical regions and reconfiguration phases as shown in Table 2.

Table 2: Mapping of the two model halves to physical regions and reconfiguration phases.

Half	Stage	Region	Phase
A	Encoder + Bottleneck	0	0
B	Decoder + Head	0	1

2.3 DFX Streamer Management for current experiments

2.3.1 DFX Streamer allocation algorithm

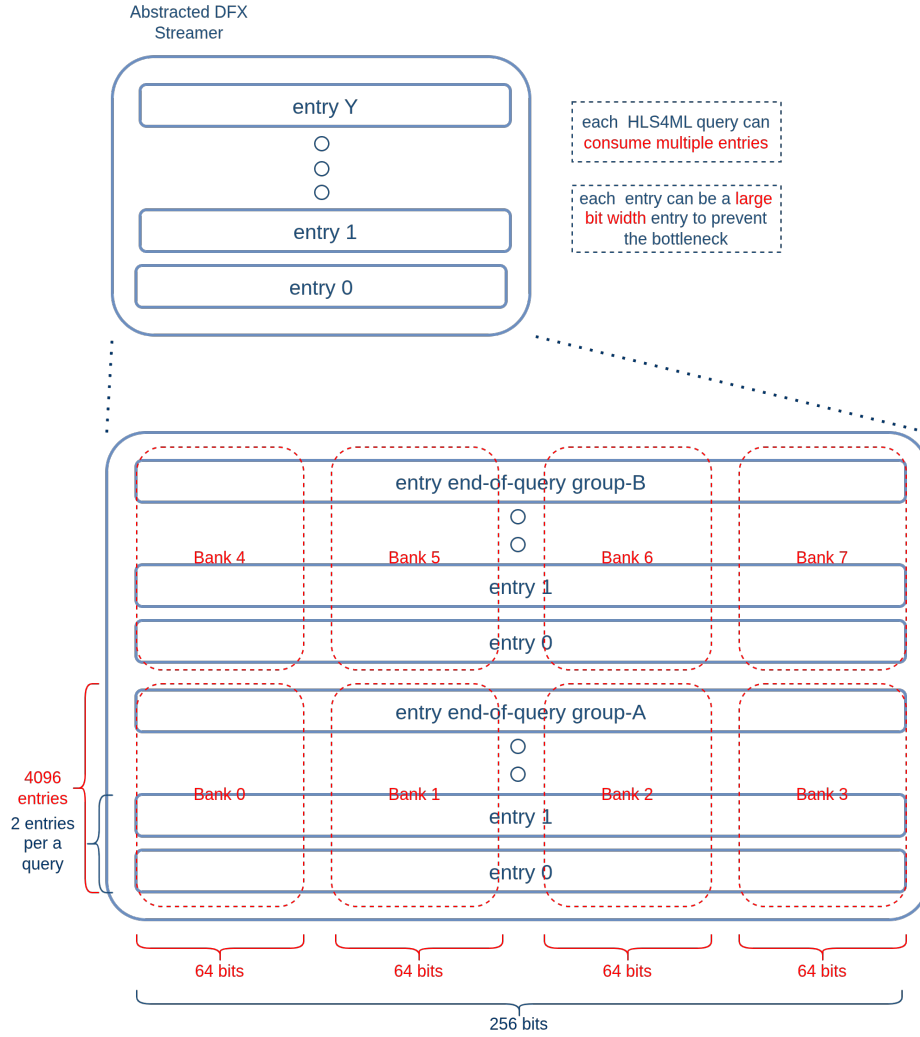


Figure 6: DFX (magic) streamer buffer allocation visualization.

Imagin the dfx streamer as an the fifo consisted of multiple entries (Fig. 6). the entry is supposed to hold the data out from the HLS4ML layer stream. For instance, if the HLS4ML stream dump (2,1,16) with 16 bits precision for each query. Therefore, the HLS4ML will dump $16 \times 16 = 256$ bits output for each cycle.

Assume that Xilinx's URAM provides 64 bits per bank; in that case, we may need at least 4 banks at the output. And the URAM can support 4096 rows therefore we can hold $4096/(2*1) = 2048$ for this set. User can have more of set the URAM to support more queries.

The overall buffer-sizing procedure is summarised in Algorithm 1. It works in four stages:

1. compute the bank geometry of every stream;
2. hand each stream a physical streamer for the phases it is alive, reusing a free streamer of the same size and region whenever possible;
3. spend the remaining bank budget on the streamer with the smallest capacity;
4. report the result.

Algorithm 1 DFX streamer buffer allocation

Require: streams (each with *shape*, *precision*, *region*, *allocPhase*, *freePhase*); bank budget N ; phase count P ; bank width BW and depth BD

Ensure: for every streamer: its bank count and buffered-query capacity

```
1: Stage 1: per-stream geometry
2: for each stream  $s$  do
3:    $entries \leftarrow$  product of all but the last dimension of  $s.shape$   $\triangleright$  items per query
4:    $bits \leftarrow s.precision \times$  (last dimension of  $s.shape$ )
5:    $s.banks \leftarrow \lceil bits/BW \rceil$   $\triangleright$  banks needed for one item
6:    $s.cap \leftarrow \lfloor BD/entries \rfloor$   $\triangleright$  queries one bank set can hold
7:   if  $s.cap = 0$  then
8:     error: “one query does not fit in a bank set”
9:   end if
10: end for

11: Stage 2: allocate streamers across phases (reuse when possible)
12:  $free \leftarrow \emptyset$ ;  $using \leftarrow \emptyset$ ;  $streamers \leftarrow \emptyset$ 
13: for  $phase \leftarrow 0$  to  $P - 1$  do
14:   for all stream  $s$  with  $s.allocPhase = phase$  do
15:     look for a free streamer  $f$  with  $f.banks = s.banks$  and  $f.region = s.region$ 
16:     if such  $f$  exists then
17:       attach  $s$  to  $f$ ;  $f.freePhase \leftarrow s.freePhase$ ; move  $f$  from  $free$  to  $using$ 
18:     else
19:       create new streamer ( $banks = s.banks$ ,  $region = s.region$ ,  $mul = 1$ , holding
20:        $s$ )
21:       add it to  $using$  and to  $streamers$ 
22:     end if
23:   end for
24:   move each  $using$  streamer whose  $freePhase = phase$  back to  $free$   $\triangleright$  data no longer
25:   needed
26: end for
27: if  $streamers = \emptyset$  then
28:   return  $\{streamers, banksUsed = 0\}$ 
29: end if

28: Stage 3: grow buffers within the bank budget (water-filling)
29: while true do
30:    $banksUsed \leftarrow \sum_{d \in streamers} d.banks \times d.mul$ 
31:    $b \leftarrow$  streamer with the smallest capacity  $d.mul \times d.cap$   $\triangleright$  the bottleneck
32:   if  $banksUsed + b.banks > N$  then
33:     break  $\triangleright$  next growth would exceed the budget
34:   end if
35:    $b.mul \leftarrow b.mul + 1$   $\triangleright$  give the bottleneck one more bank set
36: end while

37: Stage 4: report
38: return  $\{streamers, banksUsed, \min_{d \in streamers} (d.mul \times d.cap)\}$ 
```

2.3.2 4-sub model DFX streamer allocation

Running the allocation algorithm on the four-part scheme yields the streamer schedule shown in Figure 7.

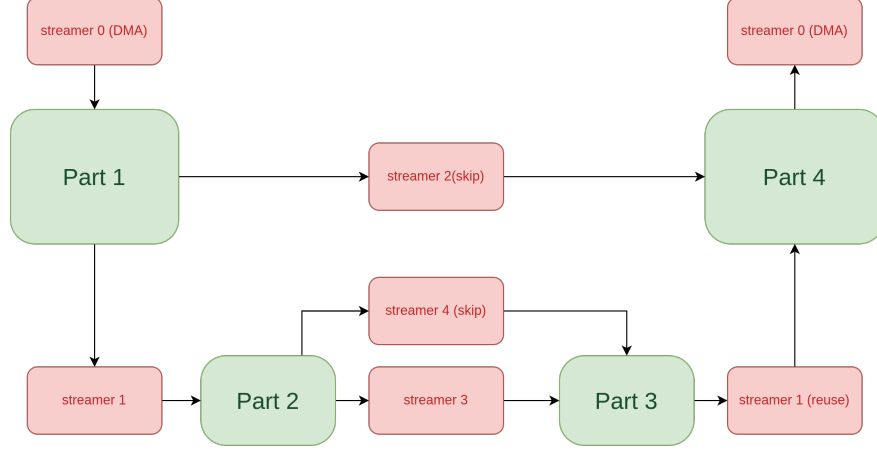


Figure 7: DFX streamer schedule for the four-part partitioning scheme.

The resulting bank usage is summarised in Table 3. Four physical streamers are created: **streamer_1** is reused by both region-0 main streams (**p0_main** then **p2_main**), while the long-lived **p0_skip** gets its own streamer with the largest number of bank sets. The total banks used, $\sum(\text{banks/set} \times \text{amt set}) = 64$, stays within the bank budget $N = 64$.

Table 3: DFX streamer bank usage for the four-part scheme.

Streamer	Region	banks/set	Free phase	amt set	banks (banks/set $\times$ amt set)
streamer_1	0	4	3	4	16
streamer_2	0	2	3	15	30
streamer_3	1	2	2	1	2
streamer_4	1	4	2	4	16
Total banks used					64

The columns of Table 3 are defined in Table 4.

Table 4: Column definitions for Table 3.

Column	Code variable	Meaning
Streamer	<code>name</code>	Identifier of the physical DFX streamer (a URAM bank group).
Region	<code>region</code>	Reconfigurable region whose data this streamer holds.
banks/set	<code>amt_banks_per_entry</code>	URAM banks in one bank set (banks needed to store one entry).
Free phase	<code>next_fin_phase</code>	Phase at which the stored data is consumed, after which the streamer may be reused.
amt set	<code>mul_factor</code>	Number of bank sets replicated for this streamer.
banks	—	Total banks used by this streamer, $\text{banks/set} \times \text{amt set}$.

Table 5 lists each inter-partition stream, the streamer it is assigned to, the entries it produces per query (`amt_entry_per_query`), the queries one bank set can hold (`amt_query_per_bankGrp`), and the resulting buffered-query capacity (`total_query` = `amt set` $\times$ `queries/set`). `p0_skip` reaches only 960 (red), making it the bottleneck on the overall buffered-query count.

Table 5: Per-stream buffering capacity for the four-part scheme.

Stream	Streamer	Entries/query	Queries/set	Total Q
p0_main	streamer_1	16	256	1024
p2_main	streamer_1	16	256	1024
p0_skip	streamer_2	64	64	960
p1_main	streamer_3	4	1024	1024
p1_skip	streamer_4	16	256	1024

2.3.3 2-sub model DFX streamer allocation

Running the allocation algorithm on the two-part scheme yields the streamer schedule shown in Figure 8.

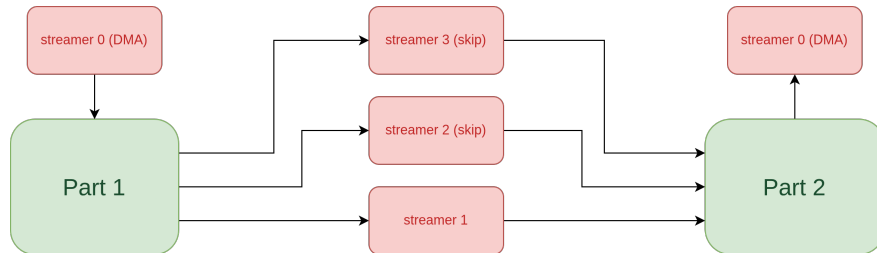


Figure 8: DFX streamer schedule for the two-part partitioning scheme.

The resulting bank usage is summarised in Table 6. Because the model is split at a single boundary, the three crossing streams (`ha_bneck`, `ha_skip2`, `ha_skip1`) are all alive at the same time, so each is given its own physical streamer and no reuse is possible. All three live

in region 0 and are freed at phase 1, once Half B has consumed them. The total banks used, $\sum(\text{banks/set} \times \text{amt set}) = 64$, exactly fills the bank budget $N = 64$. The columns have the same meaning as in Table 4.

Table 6: DFX streamer bank usage for the two-part scheme.

Streamer	Region	banks/set	Free phase	amt set	banks (banks/set $\times$ amt set)
streamer_1	0	2	1	2	4
streamer_2	0	4	1	5	20
streamer_3	0	2	1	20	40
Total banks used					64

Table 7 lists each crossing stream, the streamer it is assigned to, the entries it produces per query (`amt_entry_per_query`), the queries one bank set can hold (`amt_query_per_bankGrp`), and the resulting buffered-query capacity (`total_query` = `amt set` $\times$ `queries/set`). Both `ha_skip2` and `ha_skip1` reach only 1280 (red), making the two skip streams the joint bottleneck on the overall buffered-query count.

Table 7: Per-stream buffering capacity for the two-part scheme.

Stream	Streamer	Entries/query	Queries/set	Total Q
ha_bneck	streamer_1	4	1024	2048
ha_skip2	streamer_2	16	256	1280
ha_skip1	streamer_3	64	64	1280

3 Experiment Result

3.1 Resource Usage

3.1.1 Four-Part Scheme

The four parts are mapped onto two PR regions (`pblock_dfx_pr_region_0/1`), each region hosting two RM variants. Figure 9 reports the utilisation of each part against the device, with reference lines at the full device limit and at the 50% per-region (static region included)

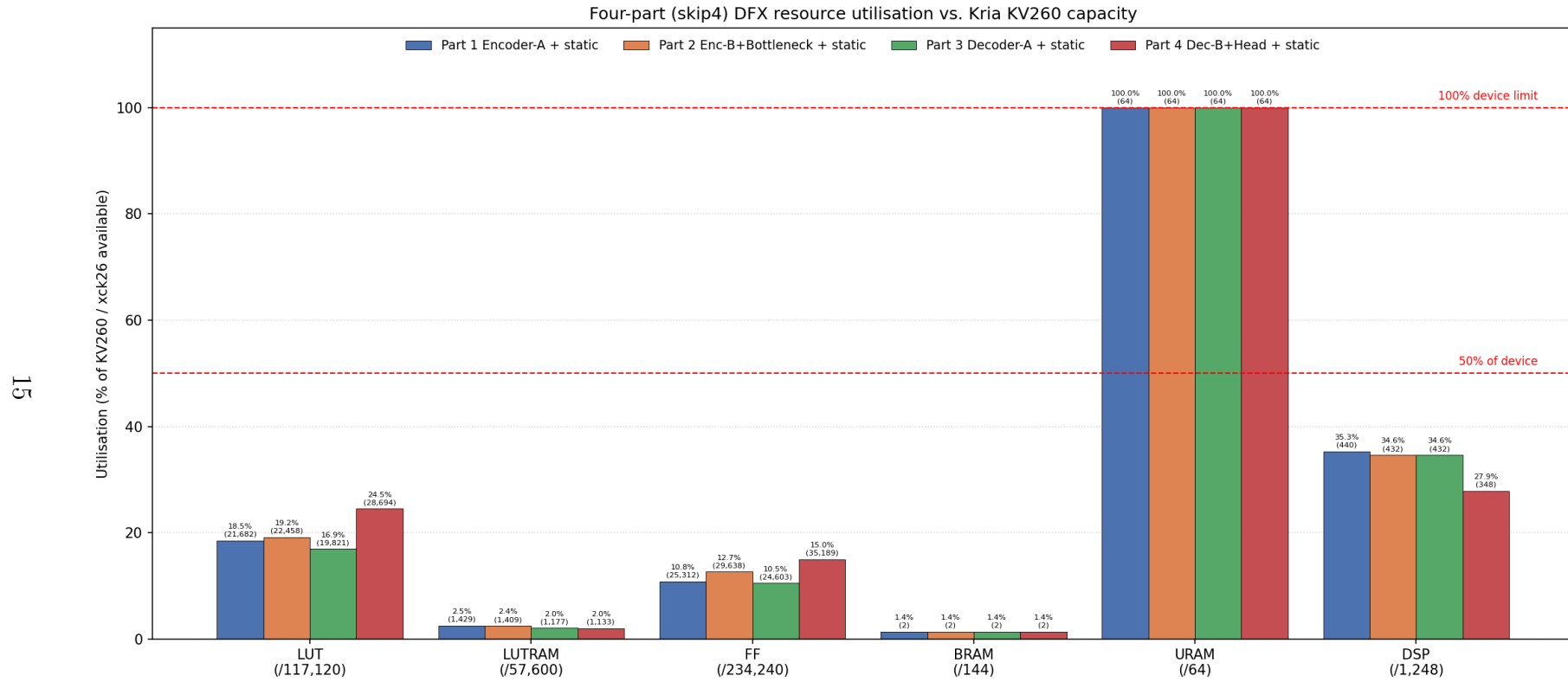
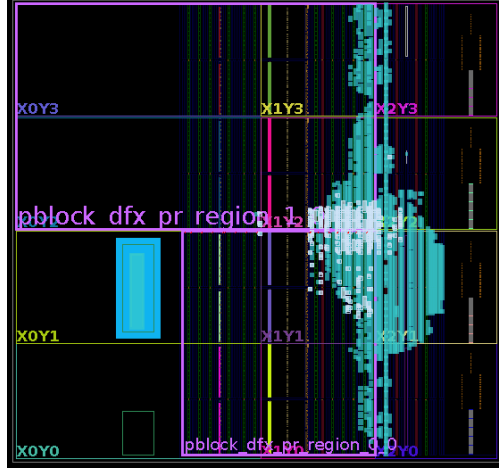


Figure 9: Four-part (skip4) resource utilisation per part, as a percentage of the KV260 capacity. Each bar is one RM plus the static region.

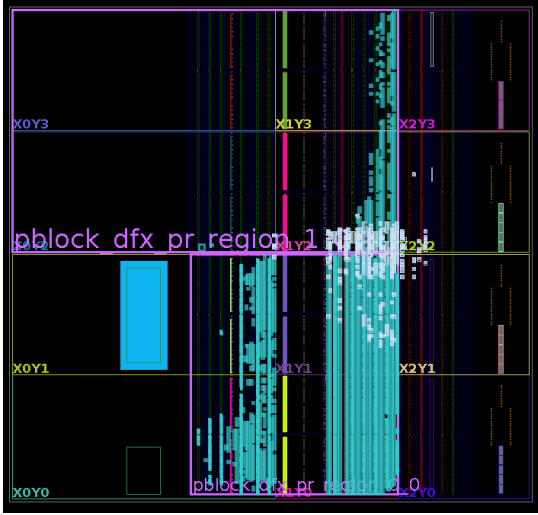
Apart from URAM (100%, static streamers), every part stays comfortably below the 50% per-region line: DSP ranges from 27.9% (348) for Part 4 to 35.3% (440) for Part 1, LUT from 16.9% (19 821) to 24.5% (28 694), and FF from 10.5% (24 603) to 15.0% (35 189); LUTRAM and BRAM stay around 2% and 1.4% respectively. The implemented device floorplans in Figure 10 show the static region (with both PR pblocks empty) and each of the four RMs routed into its region.

3.1.2 Two-Part Scheme

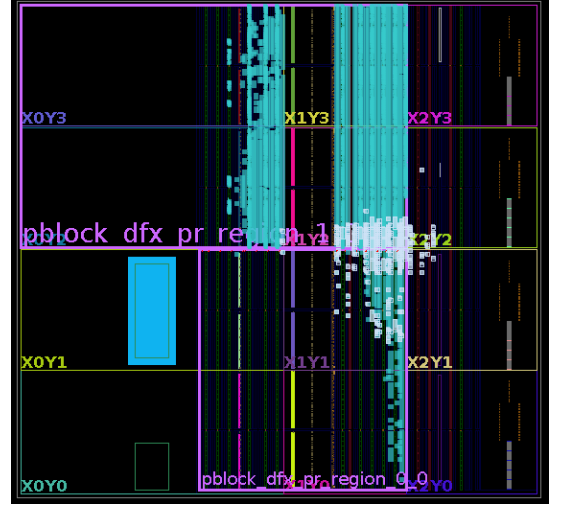
In the two-part scheme both halves share a single PR region (`pblock_dfx_pr_region_0`) over two phases. Because each half now packs two of the four parts, its per-configuration footprint is roughly double that of the four-part scheme. Figure 11 reports the utilisation of each half.



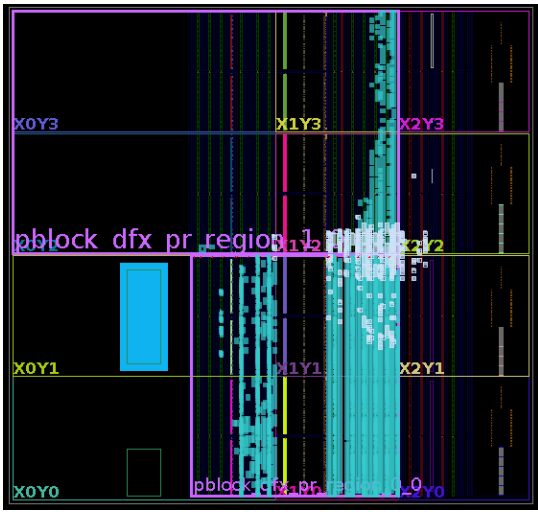
(a) Static region (both PR pblocks empty).



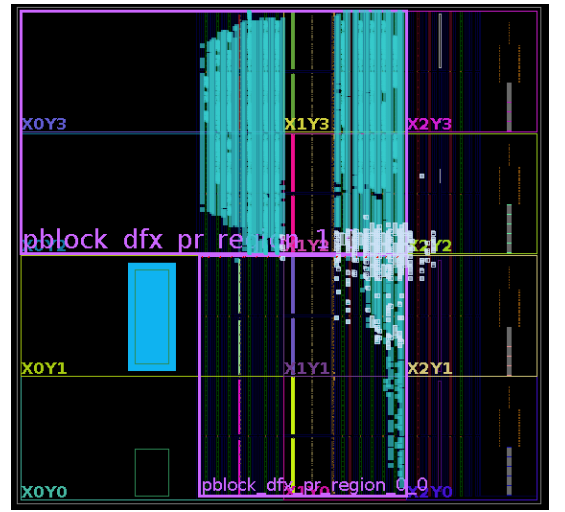
(b) Region 0 / RM 0 — Part 1 (Encoder-A).



(c) Region 1 / RM 0 — Part 2 (Encoder-B + Bottleneck).



(d) Region 0 / RM 1 — Part 3 (Decoder-A).



(e) Region 1 / RM 1 — Part 4 (Decoder-B + Head).

Figure 10: Four-part scheme: implemented ¹⁷device floorplans of the static region and the four reconfigurable modules across the two PR regions.

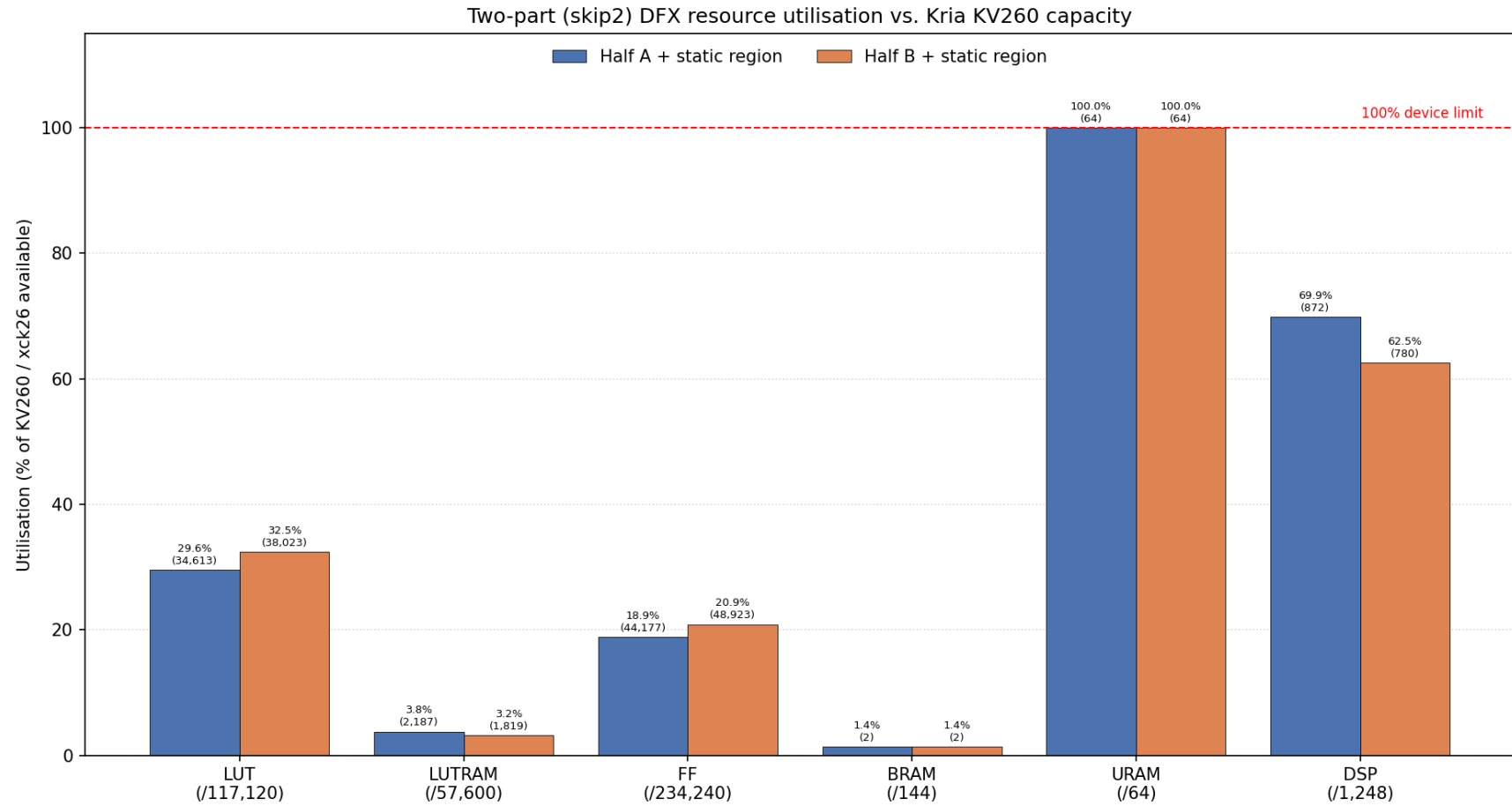


Figure 11: Two-part (skip2) resource utilisation per half, as a percentage of the KV260 capacity. Each bar is one RM plus the static region.

DSP is now the dominant logic resource, reaching 69.9% (872) for Half A and 62.5% (780) for Half B, with LUT at 29.6% (34 613) / 32.5% (38 023) and FF at 18.9% (44 177) / 20.9% (48 923); URAM remains at 100% (static streamers). Compared with the four-part scheme, folding two parts into a single region roughly doubles the LUT/FF/DSP footprint per configuration, trading fabric occupancy for fewer PR regions and fewer reconfiguration phases. The corresponding device floorplans are shown in Figure 12.

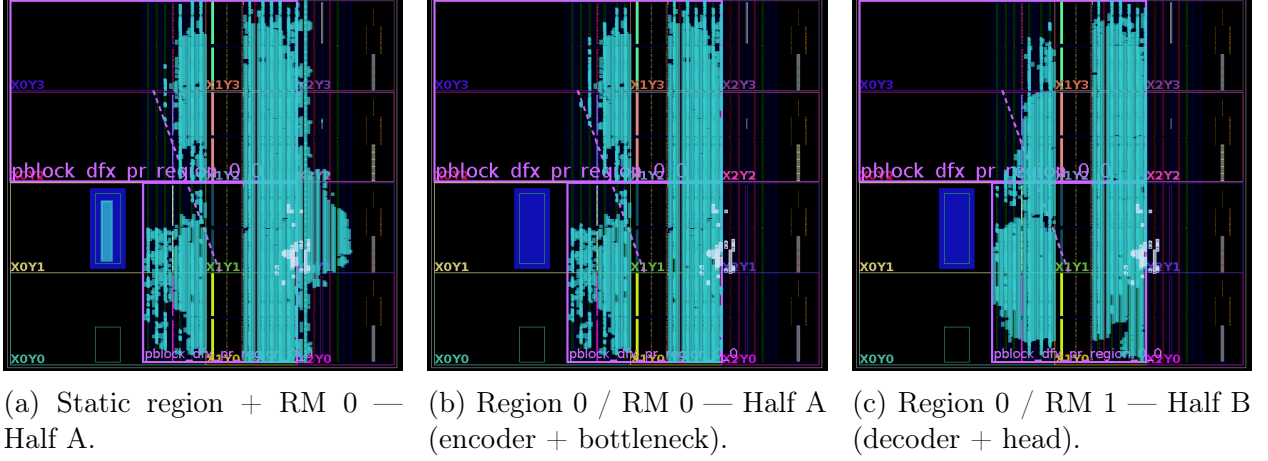


Figure 12: Two-part scheme: implemented device floorplans of the static region and the two reconfigurable modules in the single PR region.

3.2 performance

We push 900 queries into the both 2-skip and 4-skip model. The timelines below plot these two profiling counters as parallel lanes: within a slot both lanes start at the same absolute time, and the next slot starts at the later of the two end times, $t_{i+1} = t_i + \max(\text{recon_prof}_i, \text{exec_prof}_i)$. Counters are clock cycles converted to milliseconds at the build clock (≈ 100 MHz), and each box is annotated with the PR region / RM variant it drives.

3.2.1 Four-Part Scheme

With two PR regions, the four-part scheme is applying latency hiding technique : each slot reconfigures the *next* region while the *previously* loaded region is still executing, so the *recon_prof* and *exec_prof* lanes overlap in time (Figure 13).

3.2.2 Two-Part Scheme

The two-part scheme folds both halves into a single PR region over two phases, so reconfiguration and execution cannot overlap: each slot is dominated by one lane while the other is idle, yielding the serial reconfigure $\rightarrow$ execute staircase of Figure 14.

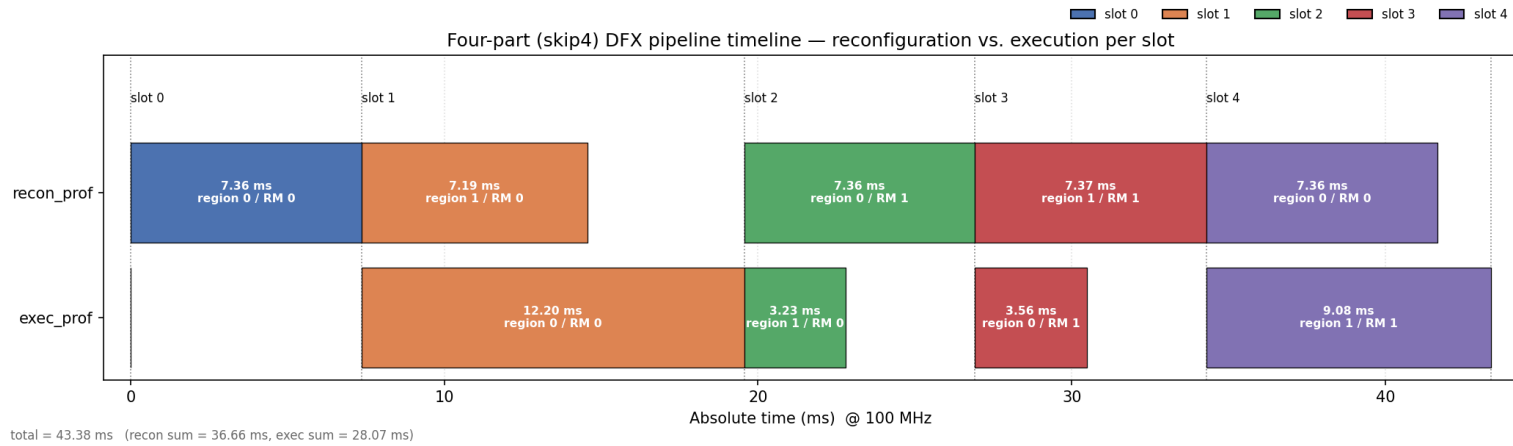


Figure 13: Four-part (skip4) DFX pipeline timeline: reconfiguration vs. execution per slot. The two regions alternate, overlapping reconfiguration of the next region with execution of the current one.

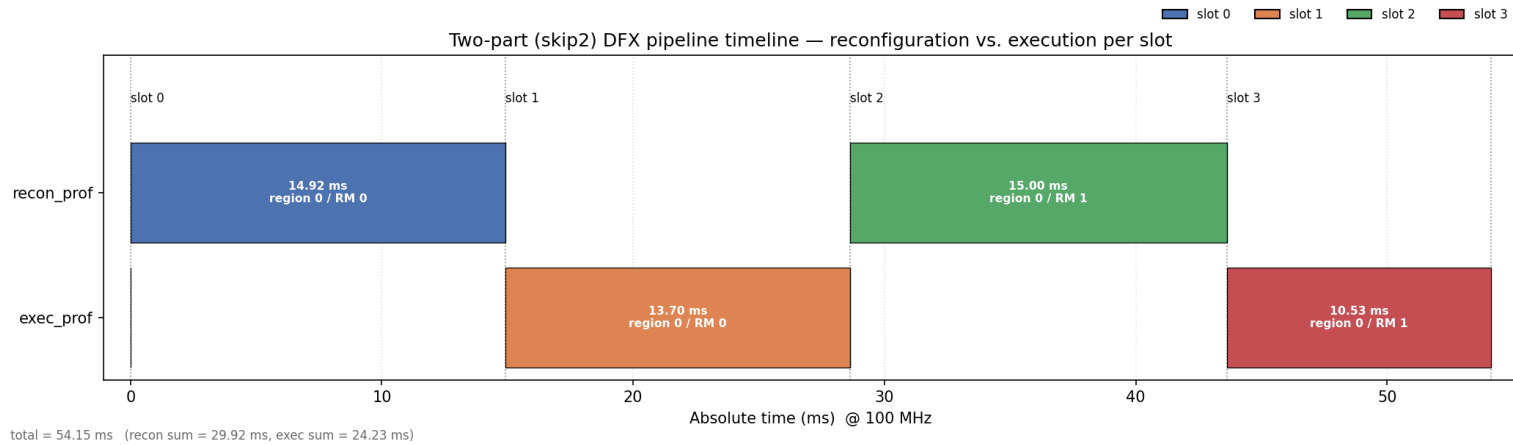


Figure 14: Two-part (skip2) DFX pipeline timeline: reconfiguration vs. execution per slot. The single shared PR region serialises the two phases.

3.2.3 Combined Comparison

Figure 15 overlays both schemes on a common time axis. The skip2 timeline is shifted so its origin aligns with the start of skip4 slot 1 (skip4 slot 0 will not be considered for performance analysis if the process have been re-execute in the next iteration), letting both schemes begin real work at the same instant. The overlap saved by latency-hiding technique is then visible directly: skip4 completes in 3601045 cycles whereas skip2 runs to 5414838 cycles.

In this calculation, we may conclude that we gain 33.5 % improvment from the ideal improvment is at 50 %. The different from the ideal case comes from the unbalance execution time in each session. Moreover, we realize that there are DDR4 memory's bandwidth competition in slot 1 (session 1).

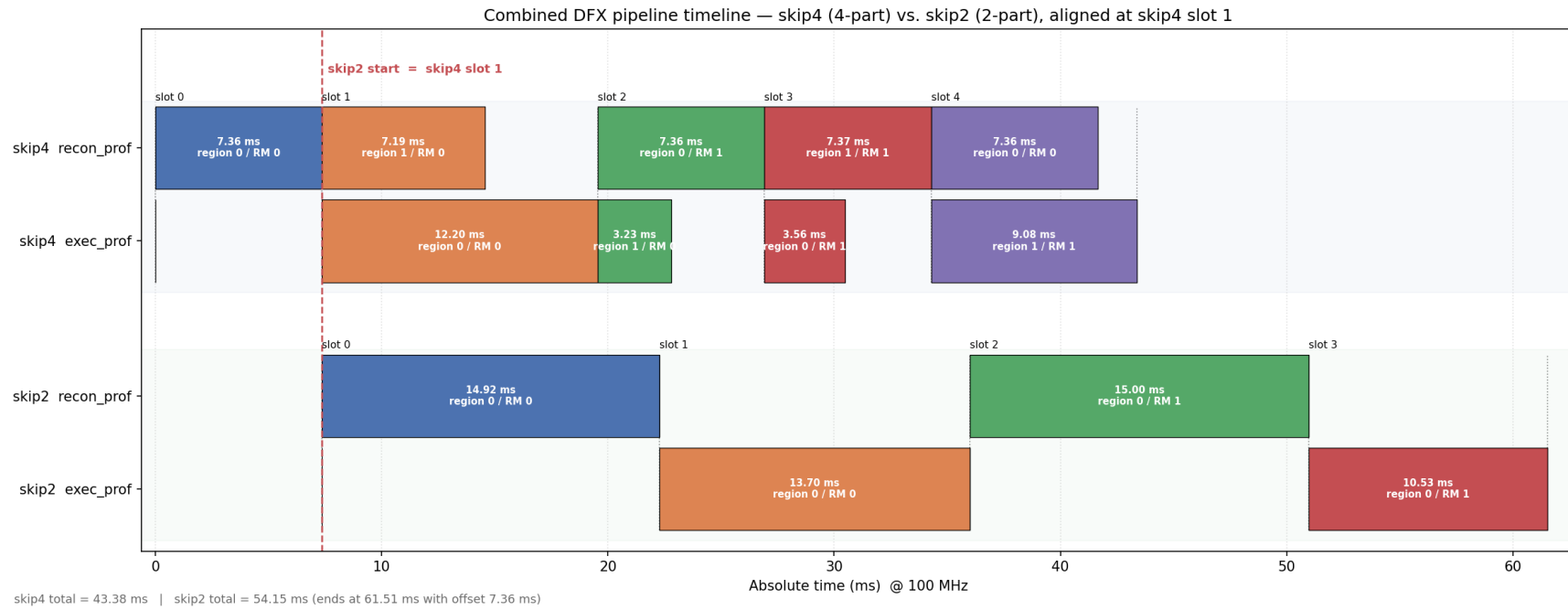


Figure 15: Combined DFX pipeline timeline: four-part (skip4) vs. two-part (skip2), aligned at skip4 slot 1. The double-buffered skip4 overlaps reconfiguration with execution, while the single-region skip2 serialises the two phases.

4 Why DFX4ML? and Its Position

We got an interesting question from my college that "WHY WE DON'T JUST REPROGRAM ENTIRE FABRIC FOR PARTIAL MODEL AND PUSH QUERIES", and use all DRAM and push many milion of queries at a time so the system will have no reconfigure overhead. (nickname: par-model on full fabric (PMFF))

Therefore, we build the table below to clarify and propose our idea. **Note that this comparison is only a conceptual proposal: the entries reflect our reasoning and expectations, and have not yet been experimentally verified.**

Table 8: Comparison criteria referenced in Table 9.

Name	Criterion
CPU usage	require CPU for operation
Fabric sharing	support unrelated system (pid control, communcation chip, network chip, etc.) operating on the same fabric
Latency	latency (round-trip time when data arrive)
Bulk exec time	overall exec time (large amount of queries)
Data-hold power	save power by holding data on BRAM/URAM
Skip output BW	support high bandwidth the data out from the model with multiple skip link

Table 9: DFX4ML (with and without the latency-hiding technique, LH) versus par-model on full fabric (PMFF). The criteria names are defined in Table 8. Green cells mark an outcome that benefits the user, yellow cells a moderate outcome, and red cells a drawback.

Criterion	DFX4ML		PMFF
	with LH	without LH	
CPU usage	No (only for initialize)		MUST
Fabric sharing	Yes		NO
Latency	Low	Moderate	High
Bulk exec time	Moderate	High	Low (if not consider DDR4 congestion)
Data-hold power	NO		NO
Skip output BW	YES		NO

We explain the key rows of Table 9 below.

- **CPU usage.** DFX4ML relies on the FPGA's self-reconfiguration, so the CPU is involved only during initialization, and even this step could be offloaded in a future upgrade so that no CPU is required at all. PMFF, in contrast, needs the CPU to reprogram the fabric every time a different partial model must be loaded.
- **Latency.** PMFF places a single partial model on the whole fabric and executes a large batch of queries at once, so it must wait until enough queries have accumulated before

it can run. This waiting inflates the round-trip latency seen by any individual query. DFX4ML instead processes each query as it arrives, keeping latency low.

- **Bulk exec time.** When a very large number of queries is processed, DFX4ML can suffer from reconfiguration overhead, because the fabric is swapped between model segments; the latency-hiding technique reduces this cost but does not remove it. PMFF performs no per-batch reconfiguration, so its bulk execution time is low — provided DDR4 bandwidth congestion is not taken into account.
- **Data-hold power.** Our earlier results show that the on-chip BRAM/URAM can hold only about 2 MB of data inside the fabric for queries buffering, whereas roughly 6 MB must be streamed from external memory for each bitstream reconfiguration. Because this data cannot be kept entirely on-chip, neither scheme can avoid external-memory traffic, and so neither saves power by holding data in BRAM/URAM.

5 Resources

The code and configurations used in this report are publicly available online:

- **Test model (hls4ml).**
<https://github.com/Tanawin1701d/hls4ml/tree/VitisUnifiedPartialExp>
- **DFX4ML — four-part (skip4) example.**
https://github.com/Tanawin1701d/dfx4ml_infra/blob/7823781813206945127c780cbfc79babe15bd354/example/multi_region_test4
- **DFX4ML — two-part (skip2) example.**
https://github.com/Tanawin1701d/dfx4ml_infra/blob/dc4e8507ed2350479570d7c86024793bd4ff18ee/example/multi_region_test5

6 Next Step

At present, integrating a model into DFX4ML is a manual process: the HLS4ML IP must be exported by hand and then imported into the DFX4ML build flow. This hand-off is workable for a single experiment, but it is tedious and error-prone, and it does not scale to larger or frequently changing models.

The next step is therefore to bridge the two projects automatically. We plan to propose a `vitisUnifiedPartial` backend — or another suitable connection mechanism — that lets HLS4ML emit reconfigurable modules directly into the DFX4ML partial-reconfiguration flow, removing the manual import step and providing a seamless path from a trained model to a self-reconfiguring DFX4ML deployment.

In parallel, we plan to upgrade the DFX Manager bank 0 register set to track the performance of the latency-hiding technique directly in hardware. By exposing on-chip counters that measure the overlap between reconfiguration and data/compute phases, we can cross-validate the latency-hiding gains.